

Exercícios — Módulo 1: Node.js

Exercício de Fixação — Node.js e módulos

1. Explique com suas palavras por que o JavaScript pode ser usado no navegador e também no Node.js, mesmo sendo ambientes diferentes.

O JavaScript é a mesma linguagem, mas o navegador e o Node.js oferecem coisas diferentes. No navegador, ele controla a interface e a interação com o usuário. No Node.js, ele roda no servidor, onde pode acessar arquivos, processar requisições e trabalhar com bancos de dados.

2. Diferencie módulo nativo, módulo de terceiros e módulo local no contexto do Node.js.

- **Módulo nativo:** já vem embutido com o Node.js, como fs, path e http.
- **Módulo de terceiros:** precisa ser instalado pelo npm, como express e nodemon.
- **Módulo local:** é um arquivo criado dentro do projeto, como um arquivo de funções ou utils.

3. Cite três situações em que o Node.js é mais adequado do que apenas JavaScript no navegador.

- Criar APIs para receber e enviar dados.
 - Ler e escrever arquivos no computador.
 - Conectar com banco de dados e manipular dados do sistema.
-

Exercício de Fixação — NPM e dependências

1. O que é o npm e qual é a sua função no desenvolvimento com Node.js?

O npm é o gerenciador de pacotes do Node.js. Ele serve para instalar, atualizar e remover bibliotecas que ajudam a desenvolver o projeto, além de gerenciar as dependências do mesmo.

2. Explique a diferença entre dependências e dependências de desenvolvimento.

- **Dependências:** são pacotes necessários para o projeto rodar em produção, como express.
- **Dependências de desenvolvimento:** são pacotes usados apenas durante o desenvolvimento, como nodemon e eslint.

3. Qual é a importância do arquivo package.json em um projeto Node.js?

O package.json guarda as informações do projeto, como nome, versão, scripts e dependências. Ele ajuda a organizar o projeto e permite que outras pessoas instalem tudo necessário com o comando npm install.

Exercício de Fixação — Runtime

1. Explique por que console.log pode existir no navegador e no Node.js, embora os dois ambientes sejam diferentes.

Porque o comando é o mesmo, e o navegador tem um terminal integrado.

2. Pesquise no REPL o resultado de `process.version` e `process.platform` e explique o que cada propriedade representa.

`process.version: 'v26.7.0'`

`process.platform: 'win32'`

3. Liste três recursos que fazem sentido no Node.js mas não são responsabilidades típicas de uma página no navegador.

- Acessar arquivos do próprio computador.
 - Consumir API.
 - Operar um banco de dados.
-

Mini Projeto Guiado — API de agendamentos

1. Qual é o objetivo do mini projeto?

O objetivo é criar uma API para controlar agendamentos de clientes, com validação de dados e regras de negócio simples, como evitar conflito de horários e garantir que o cliente só agende em datas válidas.

2. Quais funcionalidades a API deve ter?

- Cadastrar um agendamento.
- Listar todos os agendamentos.
- Buscar um agendamento por id.
- Atualizar um agendamento existente.
- Excluir um agendamento.
- Validar dados antes de salvar.

3. Quais regras de negócio devem ser implementadas?

- Não permitir dois agendamentos no mesmo horário para o mesmo usuário.
- Validar se a data e a hora são válidas.
- Não aceitar campos vazios.
- Bloquear agendamentos para serviços desativados.
- Garantir que cada agendamento tenha cliente e serviço associados.

4. Qual estrutura de pastas pode ser usada no projeto?

- `src/`
- `app.js`
- `server.js`
- `routes/`
- `controllers/`
- `services/`
- `data/`
- `utils/`

5. Como a rota de cadastro pode ser pensada?

A rota principal pode ser POST /agendamentos. Ela recebe os dados do cliente, data, hora, serviço e valida tudo antes de salvar.

6. Como a validação pode ser feita no backend?

O backend deve verificar se os campos chegaram corretamente, se a data e a hora existem, se a data não está no passado e se não há conflito com outro agendamento do mesmo usuário.

7. Como testar a API?

Pode-se usar o Postman ou o Insomnia para enviar requisições HTTP e testar as rotas, além de verificar os códigos de resposta como 200, 201, 400 e 404.

8. Qual deve ser o fluxo básico da aplicação?

- O cliente envia uma requisição para a API.
- A rota recebe a chamada.
- O controller chama a lógica de negócio.
- O serviço valida os dados.
- Se tudo estiver certo, salva o agendamento.
- A API responde com status e mensagem apropriada.

9. Qual é a entrega final esperada?

A entrega final é uma API funcional que aceita agendamentos, valida as regras de negócio e responde corretamente aos erros e às operações do CRUD, funcionando de forma organizada e reutilizável.

```
cd "C:\temp\ADS-2025-02-Desenvolvimento-Web\NODEJS\Danillo-de-Souza-Koch\node-profissional"
npm install
npm start
```

Exercício de Fixação — APIs REST e HTTP

1. Explique, com suas palavras, o que é HTTP.

HTTP é o protocolo utilizado para comunicação entre cliente e servidor na web. Ele define como as requisições e respostas são enviadas e o que cada uma significa.

2. Qual é a diferença entre uma requisição GET e uma requisição POST?

- **GET:** usado para buscar informações. Normalmente não altera os dados do servidor.
- **POST:** usado para enviar dados ao servidor, como criar um novo registro ou cadastrar algo.

3. Dê exemplo de quando a API deve responder com 400, 404 e 500.

- **400:** quando a requisição enviada está inválida, como dados faltando ou em formato errado.
 - **404:** quando o recurso solicitado não existe.
 - **500:** quando há um erro interno no servidor, como falha inesperada no processamento.
-

Exercício de Fixação — Backend e responsabilidades

1. Explique com suas palavras a diferença entre backend, API e banco de dados.

Backend é todo o processo por trás de um sistema que o usuário não vê. Banco de dados é onde se armazena os dados do sistema, e a API é usada para comunicar o backend com o externo.

2. No domínio de agendamentos, escreva três regras que devem obrigatoriamente ser garantidas no backend.

- Não aceitar mais de um agendamento do mesmo usuário no mesmo horário e mesma data.
- Aceitar apenas horários e datas válidos (exemplos inválidos: 31/02/2025, 67/22/9999).
- Cada usuário cadastrar sua própria agenda.

3. Classifique cada item como validação sintática ou regra de negócio: e-mail sem @; preço negativo; conflito de horário; CPF com quantidade errada de dígitos; serviço desativado sendo agendado.

- e-mail sem @ → validação sintática
 - preço negativo → validação sintática
 - CPF com quantidade errada de dígitos → validação sintática
 - conflito de horário → regra de negócio
 - serviço desativado sendo agendado → regra de negócio
-

Exercício de Fixação — Express.js e rotas

1. O que é o Express.js e por que ele é muito usado em Node.js?

O Express.js é um framework para Node.js que facilita a criação de APIs e aplicações web. Ele ajuda a organizar rotas, middlewares e respostas HTTP de forma simples e prática.

2. Explique a diferença entre rota e controller.

- **Rota:** define a URL e o método HTTP, como GET /usuarios.
- **Controller:** contém a lógica que será executada quando a rota for acessada.

3. Como você diferenciaria parâmetros de rota e query string em uma API?

- **Parâmetro de rota:** faz parte do caminho da URL, como /usuarios/1.
- **Query string:** vem depois do ponto de interrogação, como /usuarios?ativo=true.

4. Se você tivesse que criar uma rota para cadastrar um agendamento, qual seria a estrutura básica da URL e do método HTTP?

O mais comum seria usar o método POST em uma rota como /agendamentos, porque estamos enviando dados para criar um novo agendamento.